Building an Indirect-Geometry TOF Spectrometer in McStas

A step-by-step McStas/McStasScript guide

October 12, 2026

This practical guide introduces students to Monte Carlo neutron ray tracing and the construction of an indirect-geometry time-of-flight (TOF) spectrometer in McStas and McStasScript. Students first build a small chopper beamline, then construct a separate FANTASTIC-style model: a 2–4 Å source, uninterrupted 35 m elliptic gravity guide, sample, 30 analyser channels, and two detector tubes per channel. The exercises finish with raw powder-data reduction and exploration of preprocessed single-crystal data.

Two connected stages. Builds 1–3 use a deliberately compact teaching beamline with a pulse-shaping chopper (PSC) and bandwidth chopper (BC). Builds 4–6 start a new instrument and use the production FANTASTIC geometry, with an uninterrupted 35 m elliptic guide and no PSC or BC. Builds 7–8 use data produced by the model, the full single-crystal angular scan and its raw reduction are supplied precomputed because they are too computationally expensive for the school.

Build	Starting point	Main result
1–3	`solutions/PSC/PulseChopper.ipynb`, `solutions/BC/PSCandBC.ipynb`, then `solutions/full_primary/primary_spectometer.ipynb`	Understand chopper timing and characterize a sample-plane beam in the teaching configuration.
4–5	`solutions/one_channel/one_channel_instrument.ipynb`, then `solutions/full_instrument/full_instrument.ipynb`	Verify one analyser channel, then generate the 30-channel FANTASTIC bank.
6	Sample-model cells in `solutions/full_instrument/full_instrument.ipynb`	Compare a fixed-transfer benchmark, isotropic tabulated scattering, and a simplified phonon model.
7	`reduce_powder.ipynb`	Reduce raw McCode tube files to a powder $I(Q, \Delta E)$ map.
8	`reduction_crystal/tubes_data_reduction_crystal.ipynb`	Explore supplied, already processed single-crystal reciprocal-space and energy slices.

Contents

How to use this guide 4

1 Installation on Windows, Linux, and macOS 4

1.1 Recommended installation . . . 4

2 McStas 5

3 Indirect-geometry spectrometer 5

3.1 Monitor types . . . 7

3.2 McStas and McStasScript . . . 8

4 The McStas instrument language 9

4.1 Read the native syntax one block at a time . . . 9

4.2 Example . . . 10

4.3 Coordinate and syntax rules . . . 11

4.4 The same idea in McStasScript . . . 11

5 The main instrument components and what they do 12

5.1 Source: create the initial neutron rays . . . 12

5.2 Guide: transport and shape the beam . . . 13

5.3 Chopper: select neutron arrival times . . . 14

5.4 Arms, slits, monitors, and samples . . . 14

5.5 Analyser: select the final neutron energy . . . 15

6 Build 1: source, guide segment, and pulse-shaping chopper (PSC) 16

6.1 Lay out the primary beam . . . 16

6.2 Estimate the phase . . . 17

7 Build 2: add a bandwidth chopper (BC) 18

7.1 Add guide 2 and locate the BC . . . 18

7.2 Choose a BC harmonic and phase . . . 18

8 Build 3: complete the primary spectrometer 19

9 Build 4: make one analyser wedge (one channel) 21

9.1 Set the sample and analyser geometry . . . 22

9.2 Understand the fixed detector arm . . . 22

10 Build 5: expand to the full FANTASTIC spectrometer 23

10.1 Full-instrument settings . . . 25

10.2 Detector flight path and time of flight . . . 25

11 Build 6: add sample scattering models and verify the complete model **26**

12 Build 7: reduce raw powder TOF data to $S(Q, \omega)$ **28**

12.1 ASCII-reading workflow 29

12.2 Recover the incident wavelength from indirect TOF 30

12.3 Scattering angle and momentum transfer 30

12.4 Reference coordinate transform for a cubic A_3 scan 31

12.5 What the powder reduction should look SQW.dat input file 31

13 Build 8: explore preprocessed single-crystal data **32**

13.1 `Phonon_simple` 32

13.2 A_3 scan 32

How to use this guide

- this document explains the physics, equations, and checks behind each step.
- `indirect_geometry_spectrometer_student_workbook.ipynb` is where you complete the construction exercises.
- `solutions/PSC/PulseChopper.ipynb` and `solutions/BC/PSCandBC.ipynb` are reference solutions for the chopper lessons.
- the primary-line reference is `primary_spectometer.ipynb` in `solutions/full_primary/`.
- `one_channel_instrument.ipynb` in `solutions/one_channel/` and `full_instrument.ipynb` in `solutions/full_instrument/` are reference solutions for the final spectrometer.
- `reduce_powder.ipynb` reduces one raw powder run, while `tubes_data_reduction_crystal.ipynb` in `reduction_crystal/` lets you explore the processed single-crystal data.

What you should be able to do at the end

1. Read and write the essential blocks of a native McStas `.instr` file.
2. Build the same instrument with McStasScript and inspect the generated McStas code.
3. Predict neutron flight times, chopper gates, analyser angles, final energy, and detector arrival times before running a simulation.
4. Build and validate a source–guide–PSC instrument, add a BC, and complete a primary spectrometer.
5. Build one analyser wedge, then generate the 30-channel bank with unique component names and files.
6. Choose an appropriate sample model and explain what physical information that model produces.
7. Reduce raw McCode ASCII tube data to incident energy, energy transfer, and powder Q, then explore already processed single-crystal data in reciprocal-space and energy slices.

1 Installation on Windows, Linux, and macOS

We use Miniforge, a small conda distribution available from https://github.com/conda-forge/miniforge. Install the version for your operating system and processor, then enter the commands below in the Miniforge prompt or terminal.

1.1 Recommended installation

```
conda create -n mcstas-school -c conda-forge -c nodefaults mcstas mcstasscript
    jupyterlab numpy matplotlib pandas scipy h5py pyarrow ipywidgets mpld3
conda activate mcstas-school
conda install -c conda-forge -c scipp scipp plopp
```

Start Jupyter from the activated environment:

```
jupyter lab
```

In a notebook cell, check the Python package:

```
import mcstasscript as ms
print(ms.__version__)
import os, re
import numpy, pandas, matplotlib, scipy, h5py
import scipp, ipywidgets, plopp, mpld3, pyarrow
```

In a terminal in the same environment, check the McStas commands:

```
mcrun --version
mcdisplay --help
mcdoc Source_pulsed
```

Close Jupyter, then reopen it from the activated environment whenever it cannot find McStas or McStasScript.

2 McStas

McStas follows many neutron rays through a sequence of components. Each ray carries a position (x, y, z), velocity (v_x, v_y, v_z), time t, and statistical weight p. A component may change these values, split the ray into possible outcomes, record it, or absorb it. The simulation estimates intensity by accumulating ray weights.

A McStas instrument has four main parts: a parameter list, declarations, initialization calculations, and a trace section. The trace section places components in order. Each component has a local coordinate system and is placed with an `AT` statement relative to an earlier component or to the laboratory origin. The `ROTATED` statement sets its orientation. An instrument definition ends with `END`.

3 Indirect-geometry spectrometer

An indirect-geometry spectrometer lets a broad band of incident wavelengths reach the sample, then uses a crystal analyser to select the final wavelength. It helps to picture the experiment as four steps:

$$\boxed{\text{source and primary optics}} \longrightarrow \boxed{\text{sample}} \longrightarrow \boxed{\text{fixed-}\lambda_f\text{ analyser}} \longrightarrow \boxed{\text{time-resolved detector}}.$$

Why is this called indirect geometry? The detector does not measure the incident energy directly. Instead, Bragg's law fixes λ_f and therefore E_f, while the detector gives us an arrival time and a channel number. We subtract the known final-flight contribution from the total TOF, infer λ_i and E_i, and finally calculate $\Delta E = E_i - E_f$.

The central-path model can be written as one chain of equations. Define

$$C_v = 3956.0346\ \text{m}\,\text{Å}\,\text{s}^{-1}, \qquad C_E = 81.8042\ \text{meV}\,\text{Å}^2. \tag{1}$$

For incident path L_i, total secondary path L_f, and emission time t_0,

$$t_{\text{det}} = t_0 + \frac{L_i\lambda_i}{C_v} + \frac{L_f\lambda_f}{C_v}, \tag{2}$$

$$\lambda_i = \frac{C_v}{L_i}\left(t_{\text{det}} - t_0 - \frac{L_f\lambda_f}{C_v}\right), \tag{3}$$

$$E_i = \frac{C_E}{\lambda_i^2}, \qquad E_f = \frac{C_E}{\lambda_f^2}, \qquad \Delta E = E_i - E_f. \tag{4}$$

Quantity	Where it comes from	Why it matters
t_{det}	TOF bin in a tube monitor	Encodes emission, incident flight, and final flight
λ_f	Analyser d spacing and Bragg angle	Fixes k_f and E_f
Channel angle α	Analyser index, bank offset, and tube side	Sets the nominal final-wavevector direction
λ_i	Inverted TOF after subtracting t_0 and final flight	Gives k_i and E_i
$\Delta E, \mathbf{Q}$	Calculated from $\mathbf{k}_i - \mathbf{k}_f$	Coordinates used to interpret the scattering

Notation

λ_i and E_i are incident wavelength and energy; λ_f and E_f are the analyser-selected final values. L_i is the nominal source-to-sample path, while L_f includes sample-to-analyser and analyser-to-detector travel. The scattering vector is $\mathbf{Q} = \mathbf{k}_i - \mathbf{k}_f$. Therefore $\Delta E > 0$ means that the neutron lost energy to the sample, and $\Delta E < 0$ means that it gained energy. Distances are in metres, times in seconds inside McStas, wavelengths in angstroms, energies in meV, and wavevectors in $\mathrm{\AA}^{-1}$ unless stated otherwise.

Units

When in doubt, check the documentation for the component you are using. In this project, distances are normally metres, wavelengths are angstroms, instrument times are seconds, chopper frequencies are hertz, rotations are degrees, neutron energies are meV, and analyser mosaics are arcminutes.

3.1 Monitor types

Monitors are our windows into the simulation. Place one where you want to inspect the beam or save detector-like data. Different monitors answer different questions:

Monitor	What it records
`L_monitor`	A one-dimensional wavelength histogram. Use it to check source bands, chopper transmission, and wavelength selection.
`TOF_monitor`	A one-dimensional arrival-time histogram. It is useful for pulse timing and for checking whether events fit the detector time window.
`E_monitor`	A one-dimensional neutron-energy histogram at the monitor.
`PSD_monitor`	A two-dimensional position image, usually x versus y, for the beam footprint.
`Divergence_monitor`	A two-dimensional histogram of horizontal versus vertical ray divergence.
`Monitor_nD`	A configurable monitor that can histogram chosen ray variables, such as time, wavelength, position, or divergence, in one or two dimensions.

Each detector tube in this project is represented by a `Monitor_nD`. One component describes the cylindrical tube surface and bins both vertical position and arrival time. Here, `cylinder` creates a tube whose axis is vertical in the monitor's local frame. `xwidth` is its diameter and `yheight` is its length. The full-bank example uses two bins along local y and 1000 time bins from 20 to 45 ms. Setting `restore_neutron=1` records a ray and then lets it continue unchanged:

```
COMPONENT tube1_det0 = Monitor_nD(
    xwidth = detector_x/2, yheight = detector_y,
    restore_neutron = 1,
    options = "cylinder, y, bins=100, t, limits=[0.02 0.045], bins=1000",
    filename = "tube1_det0.dat"
)
AT (-detector_x/4 - 0.0001, 0, L2/2) RELATIVE det_pos0
```

The full bank repeats this pattern for 30 channels and two tubes per channel, always with unique component names and filenames. The ASCII file stores the histogram axes together with intensity, uncertainty, and event-count data for the reduction notebooks.

Token	Meaning in this instrument
`cylinder`	A cylindrical detector surface rather than a flat plane
`y, bins=2`	Record two bins along the local tube axis
`t, limits=[...]`	Use arrival time as the second histogram coordinate, with limits in seconds
`bins=1000`	Divide the stated time interval into 1000 bins
`restore_neutron=1`	Score the ray, restore its state, and let it continue

A monitor file is more than a rectangular block of numbers. Its header tells you the component name, shape, axes, limits, total ray count, and run parameters. In the saved full-instrument run, a tube header reports `array_2d(2, 1000)`, `xvar: y`, `yvar: TOF`, `ylabel: TOF [s]`, and `variables: I I_err N`. The first dimension is tube height and the second is time. The text data are stored as three successive blocks of 1000 rows by 2 height bins; the project reader checks this shape, reshapes to $(3, 1000, 2)$, sums intensity and event counts over height, and adds independent-bin

errors in quadrature. TOF limits in the header are edges in seconds, so compute bin centres as $t_j = t_{\min} + (j + 1/2)\Delta t$.

Location	Inspect	Ask before interpreting
Source	Wavelength and emission-time histograms	Do their axes span the requested source band and sampled time range?
After guide	PSD, wavelength, and divergence	Did guide losses change total weight, or did the guide reshape footprint and divergence?
Before/after PSC	TOF and wavelength distributions	Is the pulse narrowed, is the intended band transmitted, and what fraction of the weighted intensity remains?
Before/after BC	Wavelength and TOF distributions	Is the requested band in one identified frame, or has phase selected another opening?
At sample	PSD, divergence, wavelength, and TOF	Is the beam clipped by monitor limits, and does its footprint cover the sample?
After sample/analyser	Scattered rays and wavelength	Does the sample send rays toward the analyser, and does Bragg acceptance select the intended final band?
Detector tubes	Channel-resolved TOF histograms	Are events inside the time window, and are differences physical acceptance or counting noise?

3.2 McStas and McStasScript

McStas is both the simulation engine and the language used to describe an instrument. A `.instr` file is translated into C, compiled, and then run. McStasScript adds a Python layer that builds the same instrument from a notebook. This becomes particularly helpful when we need loops, for example, when one working analyser channel has to become 30.

A practical workflow is:

1. Open the project notebook in the Python environment that has McStasScript installed.
2. Restart the notebook kernel, then run cells from top to bottom. This avoids stale Python objects and old generated instrument definitions.
3. Start with a small neutron count and inspect the generated instrument text. Confirm the source range, component order, positions, rotations, and file paths.
4. Run the instrument, inspect monitor output, then visualize the geometry. Increase the neutron count only after the geometry and component definitions work.

```
mcrun -n 100000 instrument.instr
mcdisplay instrument.instr
mcdisplay-webgl instrument.instr
mcplot
```

The first command runs the simulation with a modest debugging count. Use `mcdisplay instrument.instr` to inspect the instrument geometry and traced neutron rays in the standard desktop viewer. Use `mcdisplay-webgl instrument.instr` for the interactive 3D browser view. The final command opens the simulation monitors. If the instrument needs non-default parameters,

append the same `name=value` arguments to either display command. Display with a small ray count while debugging, for example `mcdisplay-webgl -n 100 instrument.instr`. Exact options can differ between installations, so check `mcdisplay -help` and `mcdisplay-webgl -help`. Keep generated executables and output directories separate between simultaneous runs.

4 The McStas instrument language

The instrument language is built from only a few structural blocks. `DEFINE INSTRUMENT` names the instrument and exposes user parameters. `DECLARE` stores C variables, `INITIALIZE` performs calculations once per run, and `TRACE` lists the components in the order seen by the rays. `FINALLY` is available for optional reporting or cleanup. A small instrument often needs only `DEFINE INSTRUMENT`, `TRACE`, and `END`.

Learning goals

By the end of this section, you should be able to distinguish instrument parameters from derived variables, explain how `AT` and `ROTATED` create a coordinate hierarchy, translate one component between native McStas and McStasScript, and locate a failure in either the Python construction layer or the generated McStas layer.

4.1 Read the native syntax one block at a time

A `.instr` file is a text program, but it is not Python. McStas uses its own instrument language and allows C expressions inside declarations and code blocks. Each standard block has one clear job:

`Block or statement`	Purpose
`DEFINE INSTRUMENT`	Gives the instrument a name and declares user-settable parameters. Parameters become command-line options.
`DECLARE %{...%}`	Declares C variables used by the instrument's calculations.
`INITIALIZE %{...%}`	Calculates derived quantities once before neutron tracking begins.
`TRACE`	Begins the ordered component layout traversed by neutron rays.
`COMPONENT name = Type(...)`	Creates a named instance of a component and sets its parameters.
`AT (...) RELATIVE / ABSOLUTE`	Sets the component position in a previous component's frame or the global instrument frame.
`ROTATED (...) RELATIVE`	Sets orientation relative to a named frame; angles are in degrees.
`EXTEND %{...%}`	Runs custom C statements at a component point, for example rejecting unscattered rays.
`FINALLY %{...%}`	Optional reporting or cleanup after tracing.
`END`	Closes the instrument description.

The `%{...%}` delimiters mark embedded C code. Use semicolons after C declarations, assignments, and statements. A parameter declaration such as `double lambda_min = 2.0` has a type, name, and default value. Expressions in `INITIALIZE` can use the C math library.

The repeated form `COMPONENT guide = Guide(...)` first chooses a component type (`Guide`) and then gives this instrument instance a local name (`guide`). Parameters inside parentheses use the component's documented names. The following `AT` and `ROTATED` lines belong to that component placement. The reference after `RELATIVE` is a component instance name, not a component type. A filename parameter is a quoted string, while a value such as `Lmin = lambda_min` is an expression that refers to the instrument parameter.

4.2 Example

Uses a pulsed source, a guide segment and a wavelength monitor at the guide exit.

```
DEFINE INSTRUMENT SourceGuide(
    double lambda_min = 0.5,
    double lambda_max = 10.0,
    double guide_length = 2.0
)

TRACE
COMPONENT source = Source_pulsed(
    xwidth = 0.042, yheight = 0.030,
    Lmin = lambda_min, Lmax = lambda_max,
    t_min = 0.0, t_max = 0.004,
    dist = 0.1, focus_xw = 0.04, focus_yh = 0.04,
    freq = 50.0, t_pulse = 400e-6,
    T1 = 60.0, I1 = 1.75e12/50.0,
    tau1 = 170e-6, n_mod = 5.0,
    I_um = 3.82e10/50.0,
    tau_um = 12e-6, n_um = 5.0,
    chi_um = 0.9, kap_um = 2.2
)
AT (0, 0, 0) ABSOLUTE

COMPONENT guide = Guide(
    w1 = 0.04, h1 = 0.04,
    w2 = 0.04, h2 = 0.04,
    l = guide_length, m = 4.0
)
AT (0, 0, 0.1) RELATIVE source

COMPONENT wavelength = L_monitor(
    nL = 100, filename = "wavelength.dat",
    xmin = -0.05, xmax = 0.05,
    ymin = -0.05, ymax = 0.05,
    xwidth = 0.10, yheight = 0.10,
    Lmin = lambda_min, Lmax = lambda_max,
    restore_neutron = 1
)
AT (0, 0, guide_length) RELATIVE guide

END
```

To run:

```
mcrun -n 100000 source_guide.instr lambda_min=2 lambda_max=4
```

To find parameters supported by any component, use `mcdoc ComponentName` or the current component reference.

Stop before adding another component

Checks: the file compiles, `mcrun` creates a run directory, `wavelength.dat` has a non-zero integral, and `mcdisplay` places the guide and monitor where you expected.

4.3 Coordinate and syntax rules

- `AT (x,y,z) ABSOLUTE` positions a component in the instrument frame. `AT (x,y,z) RELATIVE previous` positions it in a previous component's coordinate frame. In most straight beamline layouts, increasing local z follows the beam.
- `ROTATED (x,y,z) RELATIVE previous` rotates the component relative to that reference frame. McStas component rotation arguments are angles in degrees. C functions such as `sin` and `cos` expect radians, so convert explicitly with `DEG2RAD`.
- Lengths are metres, wavelength values are angstroms, time is seconds, frequency is hertz, energy is usually meV, and mosaic values are arcminutes unless a component says otherwise.
- Strings in McStasScript are expressions when assigned without literal quotes, for example `guide.l = "guide_length"`. A filename is a string literal in the generated instrument, so it needs embedded quotes: `monitor.filename = 'wavelength.dat'`.
- Comments begin with `//` in the instrument language. A component name must be unique. Use unique monitor filenames for repeated channels.

4.4 The same idea in McStasScript

The equivalent notebook setup creates a Python instrument and attaches components. McStasScript properties set the instrument expressions: a string such as `"lambda_min"` means that the generated instrument should use that parameter, not a quoted text value.

```
import mcstasscript as ms

instr = ms.McStas_instr("SourceGuide")
instr.add_parameter("double", "lambda_min", value=0.5)
instr.add_parameter("double", "lambda_max", value=10.0)
instr.add_parameter("double", "guide_length", value=2.0)

source = instr.add_component("source", "Source_pulsed")
source.set_parameters(
    xwidth=0.042, yheight=0.030,
    Lmin="lambda_min", Lmax="lambda_max",
    t_min=0.0, t_max=0.004, dist=0.1,
    focus_xw=0.04, focus_yh=0.04,
    freq=50.0, t_pulse=400e-6,
    T1=60.0, I1=1.75e12/50.0,
    tau1=170e-6, n_mod=5.0,
    I_um=3.82e10/50.0,
    tau_um=12e-6, n_um=5.0,
    chi_um=0.9, kap_um=2.2
)

guide = instr.add_component(
    "guide", "Guide", AT=[0, 0, 0.1], RELATIVE="source"
)
guide.set_parameters(w1=0.04, h1=0.04, w2=0.04, h2=0.04,
                     l="guide_length", m=4.0)

wavelength = instr.add_component(
    "wavelength", "L_monitor",
    AT=[0, 0, "guide_length"], RELATIVE="guide"
)
wavelength.set_parameters(
```

```
    nL=100, filename='"wavelength.dat"',
    xmin=-0.05, xmax=0.05, ymin=-0.05, ymax=0.05,
    xwidth=0.10, yheight=0.10,
    Lmin="lambda_min", Lmax="lambda_max",
    restore_neutron=1
)

instr.settings(ncount=100000, output_path="source_guide_run")
data = instr.backengine()
ms.make_sub_plot(data)
```

The notebook approach is convenient for parameter sweeps, loops, plots, and comparisons. Underneath it all, however, McStas still compiles the generated instrument text. Open that text whenever a value or expression behaves differently from what you intended. In the project notebooks, `instr.add_parameter` declares an instrument input, `instr.add_component` creates a component instance, and `component.set_parameters` assigns several component inputs. `component.set_ROTATED` attaches an orientation to a coordinate frame. `instr.append_initialize` adds C calculations performed once before tracing; `component.append_EXTEND` adds C code at a component's trace point. Finally, `instr.settings` sets run options such as history count and output path, while `instr.backengine` generates/compiles and runs the instrument. A string such as `"lambda_min"` is emitted as an instrument expression; a Python number is a fixed value. Inspect the generated `.instr` to see the distinction.

5 The main instrument components and what they do

5.1 Source: create the initial neutron rays

The source is where every simulated ray begins. It sets the distributions of position, wavelength, emission time, and direction. Here we use `Source_pulsed`, with parameters for moderator size, wavelength limits, pulse duration, repetition rate, and pulse shape.

The de Broglie relation $p = h/\lambda = m_n v$ gives the neutron speed

$$v(\lambda) = \frac{h}{m_n \lambda} = \frac{C_v}{\lambda}, \qquad t_{\text{flight}} = \frac{L}{v} = \frac{L\lambda}{C_v}, \tag{5}$$

where C_v is defined in Eq. (1); L in metres and λ in angstroms give time in seconds. Combining de Broglie's relation with $E = p^2/(2m_n)$ gives

$$E = \frac{h^2}{2m_n \lambda^2} = \frac{C_E}{\lambda^2}. \tag{6}$$

Longer-wavelength neutrons are therefore both slower and less energetic. At 50 Hz, source pulses repeat every

$$T_s = \frac{1}{f_s} = 0.020\,\text{s} = 20\,\text{ms}. \tag{7}$$

`Source_pulsed` samples neutron wavelength, emission time, and direction from the specified distributions. The parameter t_{pulse} describes the proton-pulse interval used by the moderator model; $t_{\min}$ and $t_{\max}$ give the range sampled by the source component, not a uniform emission probability. The moderator terms shape the time distribution. Its finite width contributes directly to TOF and energy resolution because detector time alone cannot distinguish later emission from a longer or slower flight.

Each Monte Carlo history is a simulated neutron ray, not one literal neutron in the experimental beam. Its statistical weight p carries the probability or intensity represented by that history. A monitor accumulates weighted scores and estimates uncertainty from the sampled histories.

5.2 Guide: transport and shape the beam

A neutron guide is a reflective channel. Rays are reflected when they meet a wall within the wavelength-dependent acceptance of its supermirror coating. The coating value m scales the critical angle relative to natural nickel; it does not magnify the geometry or multiply the intensity. A straight guide keeps the same cross-section throughout. An elliptic guide changes shape along its length to transport a range of source positions and directions toward the sample.

There is no single "best" guide. A real design balances source phase space, wavelength band, sample footprint, divergence, available space, coating performance, and losses. Liouville's theorem gives us a useful reality check: passive optics can rearrange accepted phase space, but they cannot create brightness. More flux at the sample may come with a wider footprint or greater divergence, which can hurt resolution or background. Judge the guide with wavelength, position, and divergence monitors together. To keep the exercise manageable, `full_instrument.ipynb` uses one elliptic guide. Treat its dimensions as a starting point, not a finished engineering design.

For the FANTASTIC geometry, use an elliptic gravity guide with a 0.12 m mid-section aperture, length 35 m, and the entrance and exit dimensions in Table 1. The `dimensionsAt="mid"` option means that `xwidth` and `yheight` specify the mid-guide aperture, while `linx/liny` and `loutx/louty` describe the ellipse. The guide begins 0.5 m from the source and its exit is followed by a 1.5 m gap to the sample. The nominal axial source-to-sample path is therefore

$$L_i = 0.5 + 35.0 + 1.5 = 37.0\,\text{m}. \tag{8}$$

Table 1: FANTASTIC elliptic guide values.

Parameter	Value
Length	35 m
Mid-section aperture	0.12 m × 0.12 m
Horizontal entrance/exit dimensions	0.611 423 m / 1.682 46 m
Vertical entrance/exit dimensions	1.046 27 m / 1.945 31 m
Reflectivity R_0; coating m	0.99; 4
Gravity	enabled
Source-to-guide / guide-to-sample	0.5 m / 1.5 m

5.3 Chopper: select neutron arrival times

A disk chopper is simply a rotating clock with an opening in it. A neutron passes only if that opening crosses the beam at the right time. For one slit of angular width θ_0 and rotation frequency ν, the open fraction of a turn is $\theta_0/360^\circ$, giving the ideal open-time interval

$$w = \frac{\theta_0}{360^\circ}\frac{1}{\nu} = \frac{\theta_0}{360^\circ \nu}. \tag{9}$$

For example, a 36 degree opening at 50 Hz gives $w = 2$ ms and a 10% geometrical duty factor. This is the opening time at the chopper, not necessarily the measured pulse width downstream: moderator emission time, wavelength-dependent flight time, beam size at the disk, divergence, and histogram binning all contribute.

A chopper's `phase` is an angular setting in degrees, not a delay in seconds. In the convention used by the project components, centring an opening at desired time t_c uses

$$\phi = \mathrm{mod}(360^\circ \nu t_c, 360^\circ). \tag{10}$$

Use the source repetition period and flight times to choose the correct pulse frame. A phase error can make a chopper block the intended wavelength band.

If a source-synchronous chopper runs at harmonic q, then $\nu = q f_s$. Increasing q creates more openings in time but narrows each gate for fixed θ_0. A first feasibility test for a required interval w_{req} is

$$q \le \frac{\theta_0}{360^\circ f_s w_{\mathrm{req}}}. \tag{11}$$

In McStasScript, add and position the chopper just as you would a guide. Values calculated in the instrument's `INITIALIZE` section are assigned by their variable name as strings:

```
psc = instr.add_component(
    "PSC", "DiskChopper",
    AT=[0, 0, "chopper_distance"], RELATIVE="source"
)
psc.set_parameters(
    radius=0.25, yheight=0.065,
    theta_0="opening_PSC", nu="nu_PSC",
    nslit=1, phase="psc_phase_deg",
    abs_out=1, verbose=1
)
```

Here, `phase` is an angle rather than a time delay. Its time equivalent within one rotor period is $\phi/(360^\circ\nu)$. Keep the component origin on the beam centreline, as in the project notebook.

5.4 Arms, slits, monitors, and samples

An `Arm` is an invisible coordinate frame, it neither scatters nor absorbs neutrons. Arms give useful names to places such as the guide exit, sample, analyser centre, and detector. A `Slit`, removes rays outside a rectangular opening. A monitor records selected properties at its plane. Set `restore_neutron=1` when a ray should continue after being recorded. A sample component can change a ray's direction, energy, weight, or scattering status according to its material model.

5.5 Analyser: select the final neutron energy

The crystal analyser acts as an energy selector. It reflects only wavelengths and directions that satisfy Bragg's law within the crystal mosaic and geometrical acceptance,

$$n\lambda_f = 2d\sin A_5, \qquad A_5 = \sin^{-1}\left(\frac{\lambda_f}{2d}\right) \quad (n=1), \tag{12}$$

where d is the analyser lattice-plane spacing, n is diffraction order, and A_5 is the angle between the incident ray and the reflecting planes. The angle between the incident and reflected central rays is $2A_5$. A real solution requires $n\lambda_f \leq 2d$, otherwise the inverse sine has no physical value. The project uses first order and graphite with $d = 3.357\,\text{Å}$. Its selected final energy is

$$E_f = \frac{C_E}{\lambda_f^2} = \frac{C_E}{(2d\sin A_5)^2} \qquad (n=1). \tag{13}$$

A McStasScript analyser channel is a positioning arm followed by a monochromator. The sample-to-analyser distance sets the arm position; the rotation sets its scattering plane; the monochromator's additional $-A_5$ rotation sets the crystal Bragg orientation.

```
# Declare A5 as a double, then calculate it in INITIALIZE:
# A5 = asin(lambda_f/(2*analyser_d))*180.0/PI;
arm = instr.add_component(
    "ana_arm0", "Arm",
    AT=["d_sample_analyser*sin(DEG2RAD*channel_angle)", 0,
        "d_sample_analyser*cos(DEG2RAD*channel_angle)"],
    RELATIVE="sample_position"
)
arm.set_ROTATED([0, "channel_angle", 90],
                RELATIVE="sample_position")

mono = instr.add_component(
    "monochromator0", "Monochromator_curved",
    AT=[0, 0, 0], RELATIVE="ana_arm0"
)
mono.set_ROTATED([0, "-A5", 0], RELATIVE="ana_arm0")
mono.set_parameters(
    width=0.070, height=0.030,
    gap=0.0005, NH=11, NV=11,
    mosaich=30, mosaicv=30,
    r0=0.99, t0=1.0, DM=3.357,
    RH="analyser_RH", RV="analyser_RV", order=1
)
mono.set_GROUP("analysers")
```

For McStasScript, declare derived variables such as `A5` with `instr.add_declare_var("double", "A5")`, then add the equation to initialization using `instr.append_initialize(...)`.

The component parameter `order=1` selects first-order Bragg reflection. In the McStas component, `order=0` includes all supported diffraction orders, while a non-zero value selects a particular order. The first-order setting is useful for this idealized instrument model, but it does not model a real filter removing harmonic contamination. A real analyser can also reflect higher-order neutrons (for example, a component at $\lambda_f/2$). A cooled beryllium (Be) filter is commonly used to attenuate short-wavelength contamination while transmitting the longer-wavelength final beam. For a $\lambda_f = 4$ Å final beam, the second-order harmonic is near 2 Å.

6 Build 1: source, guide segment, and pulse-shaping chopper (PSC)

We begin with `PulseChopper.ipynb`, which contains only the source, first guide, pulse-shaping chopper (PSC), and a few monitors.

Build 1 outcome

You will place a PSC without overlapping the guide, calculate its nominal open time and phase, and demonstrate its effect with paired wavelength and TOF monitors. The accepted result is a measured change that agrees qualitatively with your timing prediction.

Build order

1. Define the source timing, wavelength range, guide length, PSC distance, opening, and frequency order.
2. Calculate the moderator timing estimate, wavelength arrival interval, gate width, and phase.
3. Add source and guide; run once without the PSC to establish the reference beam.
4. Add monitors immediately before the PSC, then add the PSC and monitors immediately after it.
5. Run with a small neutron count and compare integrated intensity as well as distribution shape.

6.1 Lay out the primary beam

Use a pulsed source followed by a 2 m straight guide. Place the guide entrance 0.1 m downstream of the source. Place the PSC 2.12 m from the source, leaving a 0.02 m gap after the guide. This makes the physical order explicit:

$$\text{source} \longrightarrow \text{guide 1} \longrightarrow \text{PSC}.$$

Add a wavelength monitor before and after the PSC. Monitors with `restore_neutron=1` record rays and then let them continue, so inserting a diagnostic does not itself remove the ray.

The following McStasScript code shows the build order. Add the source first, place the guide relative to it, then put the PSC at its source-relative distance. Assume the instrument parameters and PSC phase have already been defined:

```
source = instr.add_component("source", "Source_pulsed")
source.set_parameters(
    xwidth=0.042, yheight=0.030,
    Lmin="lambda_min", Lmax="lambda_max",
    t_min=0.0, t_max=0.004, dist=0.1,
    focus_xw=0.04, focus_yh=0.04,
    freq="source_frequency", t_pulse="pulse_length",
    T1=60.0, I1="1.75e12/source_frequency",
    tau1="moderator_tau", n_mod="moderator_n",
    I_um="3.82e10/source_frequency", tau_um=12e-6,
    n_um=5.0, chi_um=0.9, kap_um=2.2
)
guide1 = instr.add_component(
    "guide1", "Guide", AT=[0, 0, 0.1], RELATIVE="source")
guide1.set_parameters(
    w1=0.04, h1=0.04, w2=0.04, h2=0.04, l=2.0, m=4.0
```

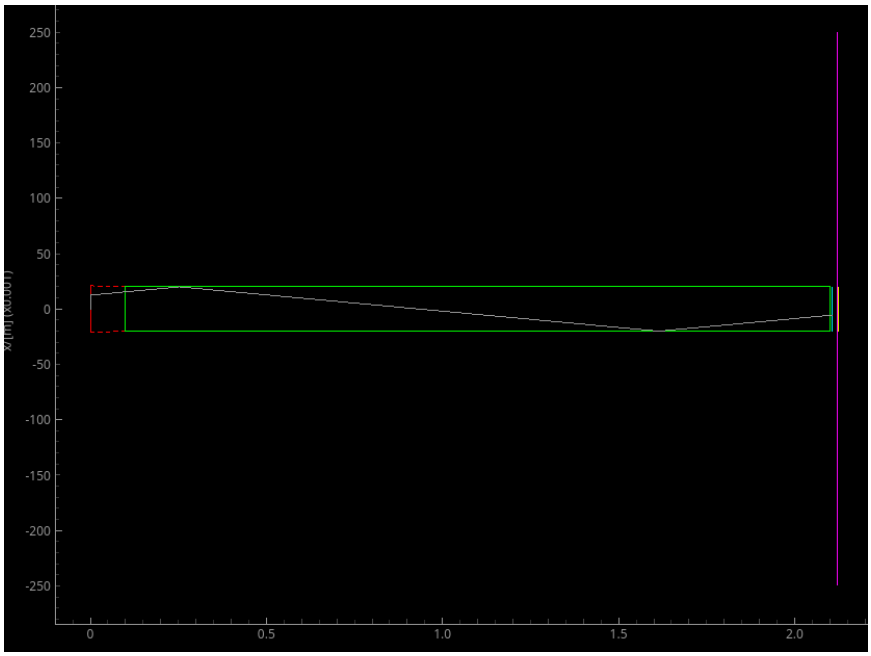


Figure 1: McDisplay side view of the Build 1 source–guide–PSC geometry.

```
)
psc = instr.add_component(
    "PSC", "DiskChopper",
    AT=[0, 0, "chopper_distance"], RELATIVE="source")
psc.set_parameters(
    radius=0.25, yheight=0.065, theta_0="opening_PSC",
    nu="nu_PSC", nslit=1, phase="psc_phase_deg", abs_out=1
)
```

In the native file, the same sequence is written as `COMPONENT source`, `COMPONENT guide1`, and `COMPONENT PSC`, each followed by its `AT` placement. Keep the guide exit just upstream of the chopper; here the 2 m guide starts at 0.1 m and ends at 2.1 m, while the PSC is centred at 2.12 m.

6.2 Estimate the phase

A useful first approximation to the moderator emission-time centre is

$$t_s \approx \frac{t_{\mathrm{pulse}}}{2} + \tau_{\mathrm{mod}} \frac{n+1}{n}. \tag{14}$$

For $t_{\mathrm{pulse}} = 400\,\mu\mathrm{s}$, $\tau_{\mathrm{mod}} = 170\,\mu\mathrm{s}$, and $n = 5$, this gives $t_s \approx 404\,\mu\mathrm{s}$. This is a timing estimate for teaching; the simulated moderator distribution is not a delta-function at t_s.

The two terms have different roles: $t_{\mathrm{pulse}}/2$ is the centre of the finite proton-pulse interval, while the second term represents the characteristic moderator delay used by this source model.

For a wavelength band $[\lambda_{\min}, \lambda_{\max}]$ and source-to-chopper path L_c, estimate the arrival interval:

$$t_{\min} = t_s + \frac{L_c \lambda_{\min}}{C_v}, \tag{15}$$

$$t_{\max} = t_s + \frac{L_c \lambda_{\max}}{C_v}, \tag{16}$$

$$t_c = \frac{t_{\min} + t_{\max}}{2}. \tag{17}$$

Then calculate the phase using Eq. (10). The nominal gate width needed to cover a source pulse and a wavelength spread $\Delta\lambda$ is

$$w_{\mathrm{req}} \approx t_{\mathrm{pulse}} + \frac{L_c \Delta\lambda}{C_v}. \tag{18}$$

Build 1: predict, run, explain

Before running `PulseChopper.ipynb`, calculate t_s, $t_{\min}$, $t_{\max}$, w_{req}, the actual gate width, and phase for the parameter values in the code cell. After the run, report the before/after monitor integrals and measured TOF FWHM. Explain any difference between the geometric gate width and the measured FWHM in terms of source time, wavelength spread, and binning.

7 Build 2: add a bandwidth chopper (BC)

Next, `PSCandBC.ipynb` adds a second guide and the bandwidth chopper (BC). The two choppers have different jobs: the PSC shapes the pulse, while the BC selects a narrower wavelength band.

7.1 Add guide 2 and locate the BC

The example uses a 10 m straight guide between choppers. With a 0.02 m clearance after each chopper, the centre-to-centre spacing is

$$L_{\mathrm{PSC,BC}} = L_{g2} + 2(0.02\,\mathrm{m}) = 10.04\,\mathrm{m}. \tag{19}$$

With $L_c = 2.12$ m from source to PSC, the source-to-BC path is $L_B = 12.16$ m. Place wavelength and TOF monitors on both sides of the BC, just as for the PSC.

In the component layout, continue from the PSC rather than rebuilding the upstream section. The 10 m guide starts 0.02 m after the PSC, the BC is placed at its downstream end with the second 0.02 m clearance:

```
guide2 = instr.add_component(
    "guide2", "Guide", AT=[0, 0, 0.02], RELATIVE="PSC")
guide2.set_parameters(
    w1=0.04, h1=0.04, w2=0.04, h2=0.04, l=10.0, m=4.0
)
bc = instr.add_component(
    "BC", "DiskChopper", AT=[0, 0, 10.02], RELATIVE="guide2")
bc.set_parameters(
    radius=0.25, yheight=0.065, theta_0=120.0,
    nu="nu_BC", nslit=1, phase="bc_phase_deg", abs_out=1
)
```

The numerical coordinate is measured from the guide entrance: 10 m guide length plus 0.02 m exit clearance. Your instrument's timing calculation supplies `nu_BC` and `bc_phase_deg`.

7.2 Choose a BC harmonic and phase

For the selected band $[2, 4]$ Å, the bandwidth is $\Delta\lambda = 2$ Å and the centre wavelength is $\lambda_c = 3$ Å. The nominal time span at the BC is

$$w_{\mathrm{BC,req}} \approx t_{\mathrm{pulse}} + \frac{L_B \Delta\lambda}{C_v} \approx 6.55\,\mathrm{ms}. \tag{20}$$

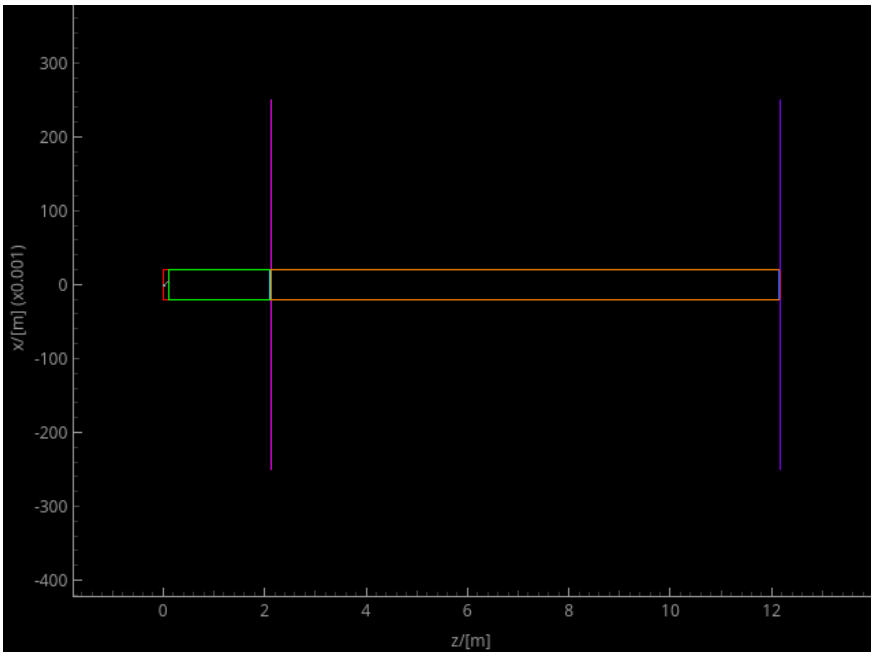


Figure 2: McDisplay side view through Build 2.

For a fixed opening $\theta_{\mathrm{BC}} = 120\,\mathrm{degree}$, estimate the largest whole-number harmonic that fits this interval:

$$q_{\mathrm{BC}} = \left\lfloor \frac{\theta_{\mathrm{BC}}}{360^\circ f_s w_{\mathrm{BC,req}}} \right\rfloor, \qquad \nu_{\mathrm{BC}} = q_{\mathrm{BC}} f_s. \tag{21}$$

Here the estimate gives $q_{\mathrm{BC}} = 1$, so $\nu_{\mathrm{BC}} = 50\,\mathrm{Hz}$ and the opening interval is 6.67 ms. Centre the BC gate at the expected arrival time of λ_c:

$$t_{c,\mathrm{BC}} = t_s + \frac{L_B \lambda_c}{C_v}, \qquad \phi_{\mathrm{BC}} = \mathrm{mod}(360^\circ \nu_{\mathrm{BC}} t_{c,\mathrm{BC}}, 360^\circ). \tag{22}$$

Using $L_B = 12.16\,\mathrm{m}$ and $\lambda_c = 3\,\text{Å}$ gives $t_{c,\mathrm{BC}} \approx 9.62\,\mathrm{ms}$ and a source-synchronous phase near 173 degree (modulo 360°). The broad 2–4 Å band spans about 6.55 ms at the BC, just inside the 6.67 ms opening.

Build 2: close the timing calculation

Calculate L_B, $w_{\mathrm{BC,req}}$, q_{BC}, ν_{BC}, the actual opening interval, and ϕ_{BC}. Then run the notebook and calculate two quantities from the monitors: the total transmitted fraction and the fraction of post-BC intensity inside 2-4 Å.

8 Build 3: complete the primary spectrometer

The notebook `primary_spectometer.ipynb` (keep the spelling used in the filename) finishes the teaching beamline by adding guide 3, a sample position, and sample-plane monitors. The path is

$$\text{source} \to \text{guide 1} \to \text{PSC} \to \text{guide 2} \to \text{BC} \to \text{guide 3} \to \text{sample}.$$

Set a guide-3 length, then place an `Arm` at the end of guide 3 and offset it by the guide-to-sample distance. This separate sample arm makes it easy to change the sample gap without changing the guide definition.

Build 3 outcome

You will finish the chopper-based primary line and characterize the beam that a sample would receive.

The final guide and sample coordinate frame can be added directly after the BC:

```
instr.add_parameter("l3", value=23.0,
                    comment="Guide 3 length [m]")
instr.add_parameter("d_guide_sample", value=1.5,
                    comment="Guide-exit to sample distance [m]")

guide3 = instr.add_component(
    "guide3", "Guide", AT=[0, 0, 0.02], RELATIVE="BC"
)
guide3.set_parameters(
    w1=0.04, h1=0.04, w2=0.04, h2=0.04,
    l="l3", m=4.0
)

sample_position = instr.add_component(
    "sample_position", "Arm",
    AT=[0, 0, "l3 + d_guide_sample"], RELATIVE="guide3"
)
```

The `AT` of a guide is its entrance. The separate `sample_position` arm is therefore placed one guide length plus the free sample gap from that entrance. Writing only `d_guide_sample` relative to `guide3` would put the sample inside the guide.

Add a wavelength monitor to check the transported band, a position-sensitive detector (PSD) to inspect the beam footprint, and a TOF monitor to inspect the arrival-time distribution. Start with broad monitor limits so that the beam is not accidentally hidden outside the plotted range.

The current primary-spectrometer notebook uses $L_3 = 23\,\mathrm{m}$ and a sample gap of $1.5\,\mathrm{m}$. With the chopper exercise values, the nominal total source-to-sample distance is

$$L_{\mathrm{sample}} = 2.12 + 10.00 + 0.04 + 0.02 + 23.00 + 1.50 = 36.68\,\mathrm{m}. \tag{23}$$

The monitor window in that notebook uses a lower limit of zero and chooses an upper limit of at least one source period, or enough time for the longest source wavelength to traverse the primary beamline:

$$t_{\max} = 10^6 \max\left(\frac{1}{f_s},\; t_{\mathrm{source,max}} + \frac{L_{\mathrm{sample}}\lambda_{\max}}{C_v}\right)\ \mu\mathrm{s}. \tag{24}$$

This completes the staged PSC–BC primary spectrometer: source, guide 1, PSC, guide 2, BC, guide 3, and sample. Save and test this instrument on its own before adding any analyser.

Build 3: characterize the sample plane

At `sample_position`, record wavelength, TOF, PSD, and divergence. For each monitor, write one sentence stating its axes and one quantitative observation. Check that the PSD is not clipped by its chosen extent and that the TOF upper limit covers the longest predicted arrival. Then change one guide parameter and identify which of the four measurements is most sensitive to it.

Primary-spectrometer handoff

Before starting the secondary spectrometer, record the source band and frequency, path lengths, sample-plane wavelength range, beam footprint, divergence range, and TOF range. For Builds

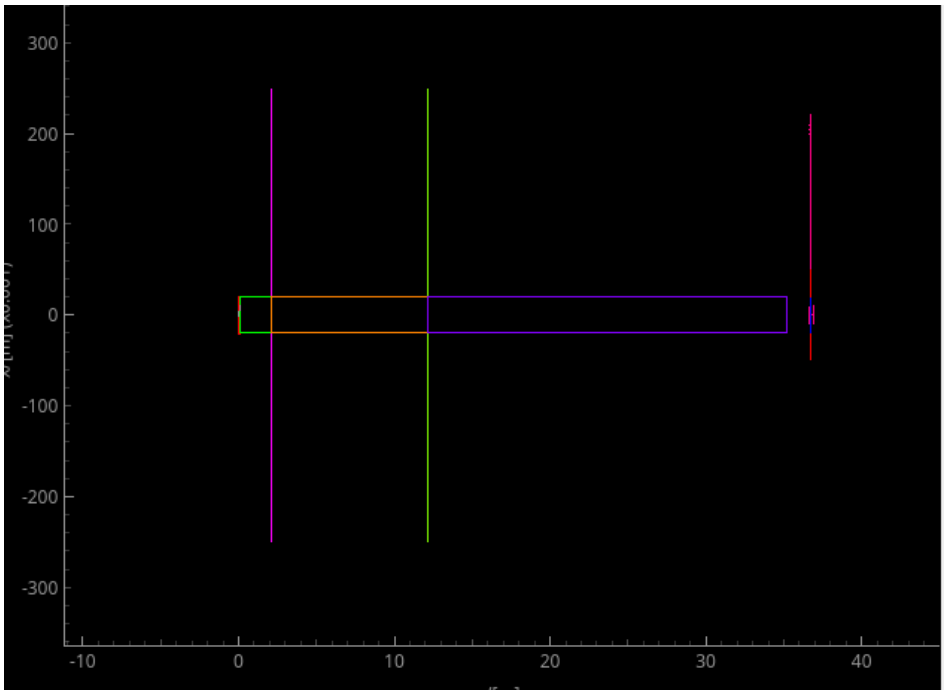


Figure 3: McDisplay view of the complete chopper-based teaching primary line, including guide 3 and the sample position.

4 and 5, start a new instrument with the production 2–4 Å source and uninterrupted 35 m elliptic guide. The analyser bank belongs to this long-guide model.

9 Build 4: make one analyser wedge (one channel)

The safest way to build the secondary spectrometer is to make one channel work first. In `one_channel_instrument.ipynb`, bring the primary beam to the sample, aim the sample model at one analyser, and then add the analyser arm, curved monochromator, fixed detector arm, slit, and two detector tubes.

Build 4 outcome

Your goal is to connect the sample, analyser, and detector coordinate systems and obtain tube counts from a ray path you can explain. This one channel is the main geometry test for the whole secondary spectrometer.

Construct one wedge in beam order

1. Calculate A_5, $2A_5$, focusing radii, the central secondary path, and an expected detector arrival interval.
2. Place the sample and focus it on one analyser face. Verify scattering before adding the analyser.
3. Place `ana_arm0` on the sample-centred arc and rotate the analyser by $-A_5$.
4. Add the fixed $-90°$ detector arm and its rolled detector-position frame.
5. Add the slit, then the two cylindrical `Monitor_nD` tubes with unique names and files.

6. Use `mcdisplay` and a short run. Only after one wedge is correct should its construction be placed inside a loop.

9.1 Set the sample and analyser geometry

Let α be the analyser's horizontal angle measured from the forward beam direction in the horizontal plane. Its sample-relative position is

$$(x, y, z) = (d_{sa} \sin\alpha,\ 0,\ d_{sa} \cos\alpha). \tag{25}$$

In McStasScript, create an `Arm` at this position, rotate it by the channel angle, and place the monochromator at the arm origin. Rotate the monochromator by $-A_5$ about its local vertical axis so its reflecting planes satisfy Bragg's law. Angles passed to component rotations are in degrees; use `DEG2RAD` when a trigonometric function in the instrument code expects radians.

For the one-channel test, use its notebook defaults, $\lambda_f = 4.2\,\text{Å}$ and $d_{sa} = 0.8\,\text{m}$, and calculate A_5 from Eq. (12). With graphite $d = 3.357\,\text{Å}$, this gives $A_5 \approx 38.72\,\text{degree}$ and reflected-beam deflection $2A_5 \approx 77.45\,\text{degree}$. The production full-bank example instead uses $\lambda_f = 4.0\,\text{Å}$ and $d_{sa} = 0.858\,\text{m}$, giving $A_5 \approx 36.57\,\text{degree}$ and $2A_5 \approx 73.14\,\text{degree}$.

9.2 Understand the fixed detector arm

The FANTASTIC geometry keeps each detector arm directed along the fixed analyser-to-detector direction. In the project, an arm is rotated by $-90°$, and the detector position is placed halfway along the fixed distance L_2. Two tubes sit side-by-side in local x. If `detector_x` is the combined width of the tube pair, each tube has width `detector_x`$/2$ and its centre is approximately

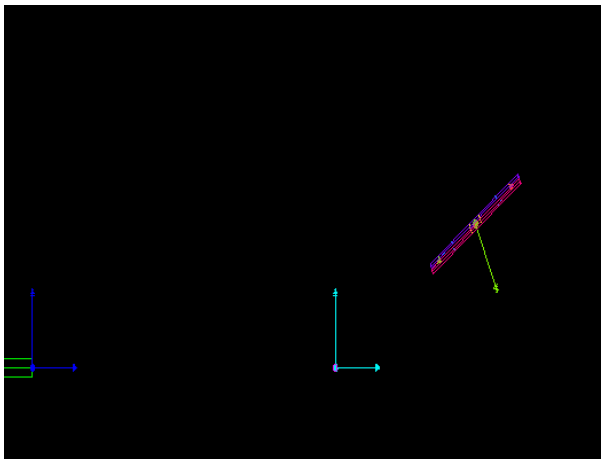
$$x_{\text{tube}} = \pm\left(\frac{\texttt{detector_x}}{4} + 0.1\,\text{mm}\right). \tag{26}$$

The small offset prevents the two detector faces from sharing an edge. A slit at the detector entrance defines the accepted beam envelope; it should be no larger than the intended tube opening.

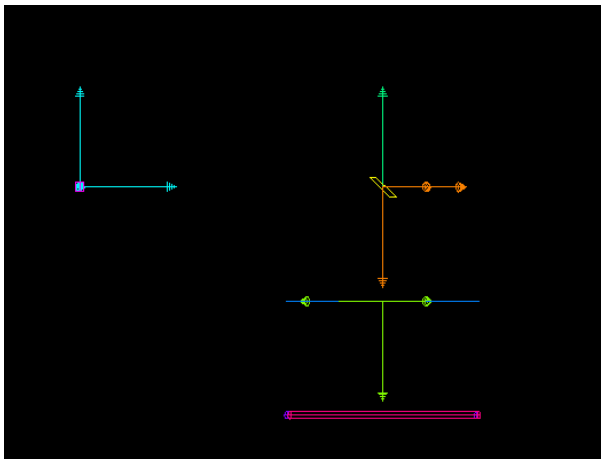
For the one-channel notebook's illustrative values $\lambda_f = 4.2\,\text{Å}$ and $L_2 = 0.5\,\text{m}$, Eqs. (12) and (30) give $A_5 \approx 38.72\,\text{degree}$, detector ray length $\ell \approx 0.512\,25\,\text{m}$, and local ray height $y_{\text{ray}} \approx -0.111\,34\,\text{m}$. The production 30-channel bank instead uses $\lambda_f = 4.0\,\text{Å}$ and $d_{sa} = 0.858\,\text{m}$. The tube centres remain at local $y = 0$ as in the FANTASTIC reference; the 0.5 m one-channel tube height covers the diagnostic central ray. The measured events still depend on the sample focus, analyser surface, slit, tube dimensions, and TOF interval.

Three angles with different jobs

The channel angle α places the analyser centre around the sample. The analyser rotation $-A_5$ orients its lattice planes. The reflected beam turns through $2A_5$.



Top view



Side view

Figure 4: McDisplay top and side views of one analyser channel. Use the top view to check the sample-centred analyser angle and the side view to check the downward reflected path and tube height; confirm the central ray with a short simulation rather than relying on the schematic alone.

10 Build 5: expand to the full FANTASTIC spectrometer

The final notebook, `full_instrument.ipynb`, turns the working wedge into a bank. It places 30 analysers at $0°, 4°, 8°, \ldots, 116°$ relative to the offset:

$$\alpha_j = \texttt{offset} + 4°j, \qquad j = 0, 1, \ldots, 29. \tag{27}$$

Build 5 outcome

You will now repeat the verified wedge without changing its physics.

Expand by component families

Create all 30 analyser arms, then all 30 monochromators, followed by all detector arms, detector-position frames, slits, tube-1 monitors, and tube-2 monitors. This ordering keeps components assigned to each McStas group contiguous. Generate the angle list first and verify that it contains 30 unique values from 0 to 116 degrees.

Name pattern	Count	Role
`ana_arm0...29`	30	Place channels on the sample-centred arc
`monochromator0...29`	30	Select and focus the final wavelength
`det_arm0...29`	30	Define the fixed reflected-beam direction
`det_pos0...29`	30	Place and roll each detector frame
`collim0...29`	30	Limit the entrance to each tube pair
`tube1_det0...29`	30	First tube in every channel
`tube2_det0...29`	30	Second tube in every channel

Before generating the bank, replace the one-channel rectangular sample focus with the full-bank focus: aim at $\texttt{offset} + 60°$, set $\texttt{focus_aw} = 120°$ and $\texttt{focus_ah} = 4°$, and use the target direction $(\sin(\texttt{offset} + 60°), 0, \cos(\texttt{offset} + 60°))$. This keeps the sample illumination matched to the complete arc.

The core McStasScript loop creates all positioning arms first, then all monochromators, so the analyser components form one contiguous group. Detector arms, detector positions, slits, and tube monitors are then created in their own loops.

```
instr.add_parameter("double", "offset", value=0.0)

for k in range(30):
    angle = f"offset + {4*k}"
    arm = instr.add_component(
        f"ana_arm{k}", "Arm",
        AT=[f"d_sample_analyser*sin(DEG2RAD*({angle}))", 0,
            f"d_sample_analyser*cos(DEG2RAD*({angle}))"],
        RELATIVE="sample_position"
    )
    arm.set_ROTATED([0, angle, 90], RELATIVE="sample_position")

for k in range(30):
    mono = instr.add_component(
        f"monochromator{k}", "Monochromator_curved",
        AT=[0, 0, 0], RELATIVE=f"ana_arm{k}"
    )
    mono.set_ROTATED([0, "-A5", 0], RELATIVE=f"ana_arm{k}")
    mono.set_parameters(
        width=0.070, height=0.030, NH=11, NV=11, gap=0.0005,
        mosaich=30, mosaicv=30, DM=3.357,
        r0=0.99, t0=1.0, RH="analyser_RH", RV="analyser_RV",
        order=1
    )
    mono.set_GROUP("analysers")
```

The following code shows the detector-side construction pattern from the project notebook. It assumes `tube_options` was defined in the generated instrument's initialization block with the desired time limits and 1000 time bins.

```
for k in range(30):
    det_arm = instr.add_component(
        f"det_arm{k}", "Arm", AT=[0, 0, 0],
        RELATIVE=f"ana_arm{k}"
    )
    det_arm.set_ROTATED([0, -90, 0], RELATIVE=f"ana_arm{k}")

for k in range(30):
    det_pos = instr.add_component(
        f"det_pos{k}", "Arm",
        AT=[0, 0, "analyser_detector_distance/2"],
        RELATIVE=f"det_arm{k}"
    )
    det_pos.set_ROTATED([0, 0, 90], RELATIVE=f"det_arm{k}")

for k in range(30):
    slit = instr.add_component(
        f"collim{k}", "Slit", AT=[0, 0, 0],
        RELATIVE=f"det_pos{k}"
    )
    slit.xwidth = "detector_x/2"
    slit.yheight = "detector_y/2"
    slit.set_GROUP("Slits")

for side, x in ((1, "-detector_x/4 - 0.0001"),
                (2, "detector_x/4 + 0.0001")):
    for k in range(30):
        tube = instr.add_component(
```

```
            f"tube{side}_det{k}", "Monitor_nD",
            AT=[x, 0, "analyser_detector_distance/2"],
            RELATIVE=f"det_pos{k}"
        )
        tube.xwidth = "detector_x/2"
        tube.yheight = "detector_y"
        tube.options = "tube_options"
        tube.filename = f'"tube{side}_det{k}.dat"'
        tube.restore_neutron = 1
        tube.set_GROUP("Detectors")
```

For each channel, create the analyser arm using Eq. (25), then attach a curved monochromator and the corresponding detector arm, detector position, collimating slit, and two tubes. In McStasScript, a Python loop can generate the repeated components. Use unique component names and unique output filenames, such as `tube1_det0.dat` through `tube1_det29.dat`. In McStas, `GROUP` is not merely a label: members form a mutually exclusive set, tried in sequence until one scatters the ray, after which the other members are skipped for that ray. The 30 monochromators are grouped as alternative analyser channels, as in the reference instrument. Keep each group's components contiguous in `TRACE`, do not insert a non-member between group members.

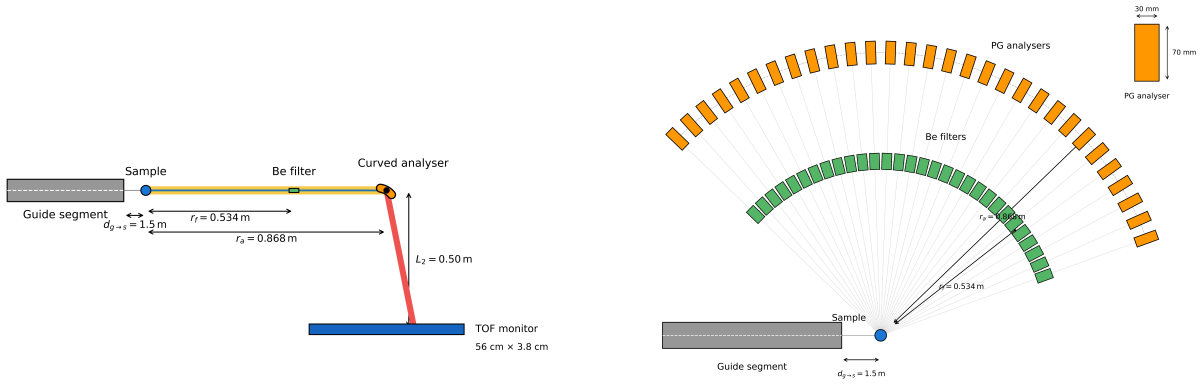


Figure 5: Secondary-spectrometer illustrations. Left: schematic side view showing the sample-to-analyser path, analyser-to-detector ray, fixed detector-arm separation L_2, and a Be filter. Right: schematic 30-channel PG analyser bank with Be-filter elements.

10.1 Full-instrument settings

10.2 Detector flight path and time of flight

One geometrical detail is easy to miss: L_2 is a fixed separation used to construct the detector arm, not the distance travelled by the reflected neutron. In the central-ray triangle, L_2 lies opposite the deflection angle $2A_5$. The analyser-to-detector ray length is therefore $\ell_{ad} = L_2/\sin(2A_5)$, and the nominal secondary path is

$$\ell_{ad} = \frac{L_2}{\sin(2A_5)}, \qquad L_f = d_{sa} + \ell_{ad}. \tag{28}$$

For $d_{sa} = 0.858\,\mathrm{m}$, $L_2 = 0.5\,\mathrm{m}$, and $2A_5 \approx 73.1$ degree, $\ell_{ad} \approx 0.523\,\mathrm{m}$ and $L_f \approx 1.381\,\mathrm{m}$. Confusing L_2 with ℓ_{ad} produces a systematic TOF error. The detector arrival time is approximately

$$t_{\mathrm{det}} = t_0 + \frac{L_i \lambda_i}{C_v} + \frac{L_f \lambda_f}{C_v}, \tag{29}$$

where t_0 is the neutron emission time, λ_i the incident wavelength, and λ_f the analyser-selected final wavelength. The first flight term varies event by event; the second is nearly fixed by the analyser

Table 2: Full FANTASTIC analyser and detector settings.

Setting	Value
Source wavelength limits	2–4 Å; no choppers in test final design
Selected final wavelength λ_f	4.0 Å
Sample-to-analyser distance d_{sa}	0.858 m
Analyser size	70 mm×30 mm
Blade count and gap	11 × 11; 0.5 mm
Horizontal/vertical mosaic	30 arcmin/30 arcmin
Analyser reflectivity/transmission	0.99 / 1.0
Fixed detector-arm separation L_2	0.5 m
Combined tube width / each tube	38.1 mm / 19.05 mm
Tube length	558.8 mm
Tube centres in local x	±9.625 mm
Tube entrance slit	19.05 mm×279.4 mm
TOF recording	20 ms–45 ms, 1000 bins
Tube-height bins	100

geometry. This is the defining indirect-geometry inversion: detector TOF is used to recover λ_i after the final-flight contribution has been removed.

The same right triangle predicts where the central reflected ray crosses the fixed detector plane. In the detector-arm frame,

$$\ell = \frac{L_2}{\sin(2A_5)}, \qquad y_{\mathrm{ray}} = -\frac{L_2}{\tan(2A_5)}. \tag{30}$$

At channel angle α, its nominal sample-relative intersection is

$$(x, y, z) = \big((d_{sa} - y_{\mathrm{ray}})\sin\alpha, -L_2, (d_{sa} - y_{\mathrm{ray}})\cos\alpha\big). \tag{31}$$

Neutron energy in meV is

$$E = \frac{C_E}{\lambda^2}, \qquad \Delta E = E_i - E_f. \tag{32}$$

11 Build 6: add sample scattering models and verify the complete model

The full-instrument notebook's default is `Isotropic_Sqw` with the supplied `SQW.dat`. It also contains switches for the other sample models: set `USE_SQW=True` and `USE_PHONON_SIMPLE=False` for `Isotropic_Sqw`; set the first to `False` and the second to `True` for `Phonon_simple`; set both to `False` for `Tunneling_sample`. These models answer different teaching questions and are not interchangeable material descriptions.

Build 6 outcome

Explain what physical signal each sample model represents, verify that the sample scatters into the analyser bank, and distinguish a synthetic timing benchmark from a tabulated or simplified material response.

Model	Select in the notebook	Appropriate use and limitation
Tunneling_sample	Both switches False	Artificial elastic/fixed-transfer benchmark, useful for checking TOF reconstruction and instrument acceptance.
Isotropic_Sqw	USE_SQW=True	Reads the supplied isotropic $S(Q,\omega)$ table; useful for powder-like energy and Q dependence but carries no crystal orientation dependence.
Phonon_simple	USE_PHONON_SIMPLE=True	Simplified fcc, nearest-neighbour, one-branch phonon model.

Here is the McStasScript code:

```
instr.add_parameter("double", "sample_E_tun", value=5.0)
instr.add_user_var("double", "sample_incident_energy")
sample_position.append_EXTEND(
    "sample_incident_energy = VS2E*(vx*vx + vy*vy + vz*vz);"
)
instr.append_initialize(
    'if (sample_E_tun < 0) { fprintf(stderr, '
    '"sample_E_tun must be nonnegative\\n"); exit(1); }'
)

sample = instr.add_component(
    "sample_tunneling", "Tunneling_sample",
    AT=[0, 0, 0], RELATIVE="sample_position"
)
sample.radius = "sample_radius"
sample.yheight = "sample_height"
sample.thickness = 0.0
sample.p_interact = 1.0
sample.f_QE = 0.0
sample.f_tun = 1.0
sample.gamma = 0.0
sample.E_tun = "sample_E_tun"
sample.target_index = 0
sample.target_x = "sin((offset + 60)*M_PI/180.0)"
sample.target_y = 0
sample.target_z = "cos((offset + 60)*M_PI/180.0)"
sample.focus_aw = 120
sample.focus_ah = 4
sample.set_ROTATED(
    ["sample_rotation_x", "sample_rotation_y", "sample_rotation_z"],
    RELATIVE="sample_position"
)
sample.append_EXTEND("""
if (!SCATTERED) { ABSORB; }
if (E_tun > 0) {
    double final_energy = VS2E*(vx*vx + vy*vy + vz*vz);
    if (final_energy >= sample_incident_energy) { ABSORB; }
    p *= 2.0;
}
""")
```

```
DECLARE
%{
double sample_incident_energy;
%}
```

```
TRACE
COMPONENT sample_position = Arm()
AT (0, 0, d_guide_sample) RELATIVE guide_out
EXTEND %{
sample_incident_energy = VS2E*(vx*vx + vy*vy + vz*vz);
%}

COMPONENT sample_tunneling = Tunneling_sample(
    radius = sample_radius, yheight = sample_height,
    p_interact = 1.0,
    f_QE = 0.0, f_tun = 1.0, gamma = 0.0,
    E_tun = sample_E_tun,
    target_x = sin((offset + 60)*M_PI/180.0),
    target_y = 0,
    target_z = cos((offset + 60)*M_PI/180.0),
    focus_aw = 120, focus_ah = 4
)
AT (0, 0, 0) RELATIVE sample_position
ROTATED (sample_rotation_x, sample_rotation_y, sample_rotation_z)
RELATIVE sample_position
EXTEND %{
if (!SCATTERED) { ABSORB; }
if (E_tun > 0) {
    double Ef = VS2E*(vx*vx + vy*vy + vz*vz);
    if (Ef >= sample_incident_energy) { ABSORB; }
    p *= 2.0;
}
%}
```

12 Build 7: reduce raw powder TOF data to $S(Q, \omega)$

This is the raw-data exercise: use `reduce_powder.ipynb` to turn a McStas run into an intensity map versus momentum transfer and energy transfer. Start with the supplied run `solutions/full_instrument/full_instrument_0/`, which contains `mccode.sim` and 60 tube files. The notebook reads its geometry and source settings from that run's metadata; do not substitute current defaults for the values saved with the data.

For the powder exercise:

1. Choose one powder run folder. Confirm that it has `mccode.sim` and the 60 tube files.
2. Read the exact source, analyser, detector, and sample settings from `mccode.sim`.
3. Read the `array_2d` shape, axis labels, and limits from each ASCII tube file. Combine the two tube-height bins and propagate their errors.
4. Calculate the TOF-bin centres and subtract the source-time and final-flight contributions to recover incident wavelength.
5. Convert incident and final wavelengths to energies; compute $\Delta E = E_i - E_f$.
6. Calculate laboratory Q_x, Q_y, Q_z and $\mathbf{Q}$.

Two different starting points

Powder: start with raw data. Use `reduce_powder.ipynb` to read `mccode.sim` and the tube `.dat` files, reconstruct the TOF kinematics, and calculate Q and ΔE.
Single crystal: start with processed data. Use `reduction_crystal/tubes_data_reduction_crystal.ipynb` with `crystal1.parquet`, `crystal2.parquet`, and `crystal3.parquet`. They already contain reciprocal-space coordinates, energy transfer, and intensity. We skip the raw single-crystal reduction because simulating and processing the full angular scan is too computationally expensive for the school session.

12.1 ASCII-reading workflow

Run McStas in text format when creating data for these exercises:

```
mcrun -n 1000000 --format=McCode full_instrument.instr
```

The run directory contains `mccode.sim`, with the parameters actually used, and detector files such as `tube1_det0.dat`. This exercise uses the ASCII output rather than NeXus. Begin with the monitor header: it identifies the `array_2d`, bin counts, axis labels, and limits. The saved header is `array_2d(100, 1000)` with tube height as the first axis and TOF as the second. In the text body, each of the three numerical blocks is stored as 1000 rows by 100 height bins: weighted intensity I, its standard error σ_I, and event count N. The limits are bin edges, so calculate the TOF-bin centres. When summing the independent height-bin estimates, add intensities and counts directly and propagate standard errors in quadrature.

```
from pathlib import Path
import io
import numpy as np
import re

run_dir = Path("solutions/full_instrument/full_instrument_0")
path = run_dir / "tube1_det0.dat"
text = path.read_text()
shape = re.search(r"^# type: array_2d\((\d+),\s*(\d+)\)", text, re.M)
limits = re.search(r"^# xylimits:\s*(.*)", text, re.M)
assert shape and limits
ny, nt = map(int, shape.groups())
ymin, ymax, tmin, tmax = map(float, limits.group(1).split())
assert "# xlabel: y [m]" in text and "# ylabel: TOF [s]" in text
values = np.loadtxt(io.StringIO(text), ndmin=2)
assert values.shape == (3*nt, ny)
intensity_y_t, error_y_t, events_y_t = values.reshape(3, nt, ny)

# Sum over the two height bins; retain independent-bin uncertainties.
dt = (tmax - tmin)/nt
tof_s = tmin + (np.arange(nt) + 0.5)*dt
intensity_t = intensity_y_t.sum(axis=1)
error_t = np.sqrt((error_y_t**2).sum(axis=1))
events_t = events_y_t.sum(axis=1)
```

12.2 Recover the incident wavelength from indirect TOF

For the final FANTASTIC guide, the nominal primary path is $L_i = 0.5 + 35 + 1.5 = 37\,\mathrm{m}$. For a legacy chopper-path run, the reduction reads its saved geometry and uses

$$L_i = \texttt{chopper_distance} + \texttt{guide2_length} + 0.06\,\mathrm{m} + \texttt{l3} + \texttt{d_guide_sample}, \tag{33}$$

where the 0.06 m accounts for three 0.02 m gaps. The names in this expression match the keys saved in `mccode.sim`. Calculate the analyser angle and secondary path from the saved parameters:

$$A_5 = \arcsin\left(\frac{\lambda_f}{2d_{\mathrm{analyser}}}\right), \qquad L_f = d_{sa} + \frac{L_2}{\sin(2A_5)}. \tag{34}$$

The emission-centre estimate from the source notebook is

$$t_0 = \frac{t_{\mathrm{pulse}}}{2} + \tau_{\mathrm{moderator}}\frac{n_{\mathrm{moderator}} + 1}{n_{\mathrm{moderator}}}. \tag{35}$$

For the project source, $t_0 = 404\,\mu\mathrm{s}$. The source's sampled interval (0–4 ms) is an allowed time window, not its mean emission time. Treating every event as if it were emitted at t_0 converts source-time spread into a wavelength/energy broadening and may also shift a reconstructed peak if the estimate is biased. A quantitative instrument would propagate or calibrate the emission-time distribution rather than regard t_0 as exact.

The analyser sets λ_f, and detector time includes the source time plus both paths:

$$t_{\mathrm{det}} = t_0 + \frac{L_i\lambda_i}{C_v} + \frac{L_f\lambda_f}{C_v}. \tag{36}$$

Now take each TOF-bin centre and solve the same equation for the incident wavelength:

$$t_i = t_{\mathrm{det}} - t_0 - \frac{L_f\lambda_f}{C_v}, \qquad \lambda_i = \frac{C_v t_i}{L_i}. \tag{37}$$

Convert the wavelengths to energies and energy transfer:

$$E_i = \frac{C_E}{\lambda_i^2}, \qquad E_f = \frac{C_E}{\lambda_f^2}, \qquad \Delta E = E_i - E_f. \tag{38}$$

12.3 Scattering angle and momentum transfer

The full bank's nominal channel angle is $\alpha_k = \texttt{offset} + 4k^\circ$, for $k = 0, \ldots, 29$. The one-channel legacy parameter `channel_angle` does not set these full-bank positions. For the project tube layout, tube 1 lies on the positive horizontal side and tube 2 on the negative side. A small central-ray correction is

$$b = \frac{\texttt{detector_x}}{4} + 0.1\,\mathrm{mm}, \qquad \delta = \arctan\left(\frac{b}{L_f}\right), \qquad \alpha = \begin{cases} \alpha_k + \delta, & \text{tube 1,} \\ \alpha_k - \delta, & \text{tube 2.} \end{cases} \tag{39}$$

This is a nominal small-angle correction, not a reconstruction of the finite sample, analyser mosaic, vertical divergence, or tube-cylinder intersection.

The incident and final wavevector magnitudes and energy transfer are

$$k_i = \frac{2\pi}{\lambda_i}, \qquad k_f = \frac{2\pi}{\lambda_f}, \qquad \Delta E = E_i - E_f. \tag{40}$$

With $+z$ along the incident beam and $+x$ toward increasing channel angle, the horizontal-plane central rays are

$$\mathbf{k}_i = (0, 0, k_i), \qquad \mathbf{k}_f = (k_f \sin\alpha, 0, k_f \cos\alpha). \tag{41}$$

Using the convention $\mathbf{Q} = \mathbf{k}_i - \mathbf{k}_f$ therefore gives

$$Q_x = -k_f \sin\alpha, \qquad Q_y = 0, \qquad Q_z = k_i - k_f \cos\alpha, \tag{42}$$

and

$$Q = \sqrt{Q_x^2 + Q_z^2} = \sqrt{k_i^2 + k_f^2 - 2k_i k_f \cos\alpha}. \tag{43}$$

12.4 Reference coordinate transform for a cubic A_3 scan

At $A_3 = 0$, assume the cubic axes are mounted as [100] $\parallel +x$, [010] $\parallel +y$, and [001] $\parallel +z$. That is a deliberate mounting choice, not a consequence of cubic symmetry. We also set `sample_rotation_x=0` and `sample_rotation_z=0`, with A_3 taken from `sample_rotation_y`.

Undo the y rotation with $\phi = A_3\pi/180$:

$$q_x = \cos\phi\, Q_x - \sin\phi\, Q_z, \qquad q_y = Q_y, \qquad q_z = \sin\phi\, Q_x + \cos\phi\, Q_z. \tag{44}$$

The operation preserves magnitude, $q_x^2 + q_y^2 + q_z^2 = Q_x^2 + Q_y^2 + Q_z^2$. If the cubic lattice constant a is known, optional reciprocal-lattice units are

$$h = \frac{aq_x}{2\pi}, \qquad k = \frac{aq_y}{2\pi}, \qquad l = \frac{aq_z}{2\pi}. \tag{45}$$

12.5 What the powder reduction should look SQW.dat input file

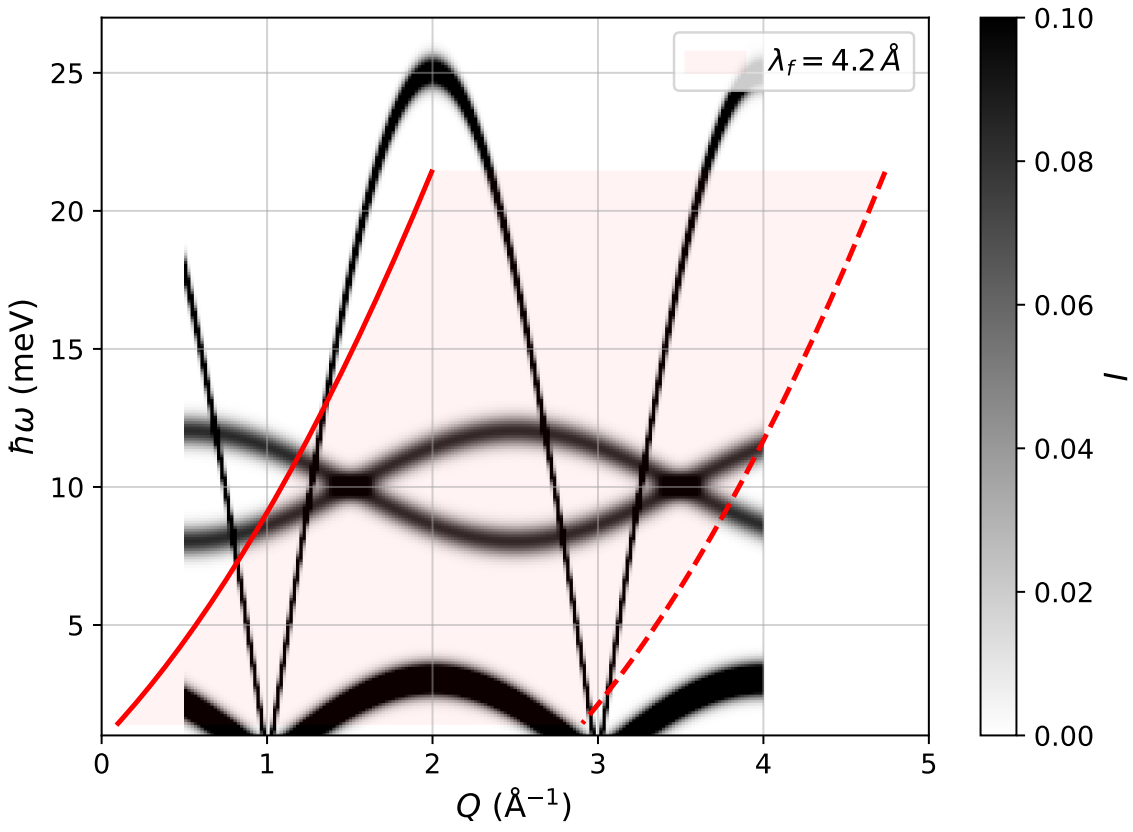


Figure 6: Reference powder-reduction map.

13 Build 8: explore preprocessed single-crystal data

Activate the Jupyter environment and install the dependencies listed in the preceding single-crystal preparation subsection. Open `reduction_crystal/tubes_data_reduction_crystal.ipynb` from `reduction_crystal/`.

13.1 `Phonon_simple`

Parameter	Notebook value	Meaning
`radius`, `yheight`	0.01 m, 0.01 m	Simulation sample geometry
`sigma_abs`	0.171 barn/atom	Absorption
`sigma_inc`	0.003 barn/atom	Incoherent cross section
`a`	4.9396 Å	fcc lattice parameter
`b`	9.405 fm	Coherent scattering length
`M`	207.2 u	Conventional atomic mass
`c`	12.90095 meV Å	Effective sound coefficient $\hbar v_s$
`DW`, `T`	1.0, 300 K	Reference factor and temperature

```
sample_phonon.sigma_abs = 0.171
sample_phonon.sigma_inc = 0.003
sample_phonon.a = 4.9396
sample_phonon.b = 9.405
sample_phonon.M = 207.2
sample_phonon.c = 12.90095
sample_phonon.DW = 1.0
sample_phonon.T = 300.0
sample_phonon.target_x = "sin((offset + 60)*M_PI/180.0)"
sample_phonon.target_y = 0.0
sample_phonon.target_z = "cos((offset + 60)*M_PI/180.0)"
sample_phonon.focus_aw = 120.0
sample_phonon.focus_ah = 4.0
```

13.2 A_3 scan

The instrument exposes the sample rotations as `sample_rotation_x`, `sample_rotation_y`, and `sample_rotation_z`. In this guide, A_3 is the `sample_rotation_y` scan angle, in degrees:

$$A_3[^\circ] = \texttt{sample_rotation_y}. \tag{46}$$

Keep `sample_rotation_x`=0 and `sample_rotation_z`=0. In a native McStas scan, `-n` is the number of neutron histories simulated at each angle and `-N` is the number of scan points. For reference, the underlying command has the form

```
mcrun -n 1000000 -N 100 --format=McCode --dir=A3_scan \
  full_instrument.instr sample_rotation_x=0 \
  sample_rotation_y=0,360 sample_rotation_z=0
```

Single-crystal simulation and reduction are precomputed

A useful scan needs many sample angles, a large ray count at every angle, 60 detector files per point, and a sizeable reduction step afterwards. That is too computationally expensive to complete during this school. We have therefore done the heavy processing in advance and

supplied `reduction_crystal/crystal1.parquet`, `reduction_crystal/crystal2.parquet`, and `reduction_crystal/crystal3.parquet`. Use these files in class rather than trying to regenerate the raw scan.

Python dependencies for the processed single-crystal notebook

Activate the same environment that launches Jupyter, then install the notebook's third-party dependencies:

```
conda activate mcstas-school
conda install -c conda-forge pandas numpy matplotlib scipy h5py ipywidgets mpld3 pyarrow
conda install -c conda-forge -c scipp scipp plopp
```

The modules `os` and `re` belong to Python's standard library and do not need separate installation. The remaining imports come from these packages:

Import name	Purpose in the notebook
`pandas, numpy`	Tabular data and numerical arrays
`matplotlib`	Static plotting and colour normalization
`scipy`	Numerical utilities, including `gaussian_filter`
`h5py`	HDF5 support imported by the notebook
`scipp`	Labelled, unit-aware arrays and binning
`ipywidgets`	Interactive sliders and notebook controls
`plopp`	Scipp-oriented interactive plotting widgets
`mpld3`	Matplotlib-to-browser plotting support
`pyarrow`	Reading and concatenating the Parquet tables